

Docker: первый контейнер

Реферат по Session 13

30 сентября 2026 • Модуль 03 • Ubuntu ARM64 в UTM

Задача и результат

Я запустил контейнерный Nginx, проверил публикацию порта, HTTP, журнал запроса и остановку/повторный запуск того же экземпляра. Практика и короткая защита зачтены с подсказками; это не экзамен на самостоятельное администрирование Docker.

По скриншотам: Docker Client и Server 29.1.3, linux/arm64. Контейнер s18-web, образ nginx:1.30.5-alpine, ID 0e4c538a2d84. Публикация 127.0.0.1:8133 -> 80/tcp. HTTP 200, заголовок nginx/1.30.5 и страница Welcome to nginx!.

В тикете предполагалось имя s13-web, но я ввёл s18-web. Это не другая лабораторная и не ошибка приложения: в отчёте сохраняем фактическое имя.

Образ, контейнер, процесс

Образ - шаблон файлов и настроек запуска. Контейнер - созданный экземпляр с ID, конфигурацией и записываемым слоем. Он может быть остановлен. Работающий контейнер - этот экземпляр с запущенным процессом.

Linux-контейнеры используют ядро хоста; здесь хост - Ubuntu VM. Контейнер не является отдельной полноценной виртуальной машиной. Два контейнера одного образа независимы в жизненном цикле, хотя могут зависеть от общих внешних сервисов.

Доступ к Docker

Без sudo команда показала Client, затем permission denied к docker.sock. С sudo появился Server. Это доказало проблему доступа пользователя, а не необходимость переустановки Docker. Права сокета и группы ради обхода я не менял.

Как читался запуск

```
sudo docker run -d --name s18-web \
  -p 127.0.0.1:8133:80 nginx:1.30.5-alpine
```

Историческая команда, не запускать повторно при существующем имени. run создаёт новый контейнер; -d оставляет его работать в фоне; --name задаёт имя; -p публикует порт. Образа локально не было: Docker скачал его и напечатал ID. Сообщение Unable to find image locally здесь не было финальной ошибкой.

02 / Адрес, версия и проверка

Что я подтвердил

```
curl в Ubuntu -> 127.0.0.1:8133 -> контейнер:80 -> Nginx
```

8133 принадлежит публикации на Ubuntu; 80 - приложению внутри контейнера. Это не перенастройка Nginx на 8133. localhost Mac относится к Mac, поэтому запрос выполнялся из SSH-сеанса Ubuntu.

Мои наблюдения

1. ps -a до запуска пуст; ss для 8133 не показывает listener.
2. После run: ps показывает Up и 127.0.0.1:8133->80/tcp.
3. curl: HTTP 200 и нужная HTML-страница, не просто открытый порт.
4. logs: строка GET / HTTP/1.1 с 200, размером 896 и curl/8.5.0.
5. Время запроса 12:45:40 UTC совпадает с HTTP Date; запись связывает ответ с контейнером.

В журнале адрес клиента виден как 172.17.0.1. Адрес в контейнерной сети может отличаться от адреса назначения curl из-за пути через Docker. Это не повод менять публикацию или объявлять запрос посторонним.

Почему я выбрал конкретный тег

Я предложил не использовать изменяемый stable-alpine для воспроизводимого развёртывания. Я выбрал nginx:1.30.5-alpine; скачивание и работа подтверждены на VM. Этот отчёт не объявляет выбранную версию текущей рекомендацией для production.

Тег указывает вариант образа, но не является криптографической гарантией неизменности. Digest фиксирует содержимое. Перезапуск существующего контейнера не обновляет образ автоматически. В этой первой практике достаточно конкретного тега; автоматические обновления и закрепление по digest не внедрялись.

Чего я не делал

Я не создавал Dockerfile или Compose, не собирал свой образ, не настраивал volumes и registry, не менял firewall и не добавлял пользователя в docker-группу. На дату этой практики пользователи и роли были следующим шагом. Группа docker не является ограниченной ролью оператора: стандартный доступ к daemon даёт root-уровень возможностей.

03 / Остановка не равна удалению

Жизненный цикл и защита

Эксперимент

После `docker stop s18-web`:

- `curl` к 8133 перестал подключаться: ошибка (7).
- `ps -a` сохранил контейнер `0e4c538a2d84` со статусом `Exited (0)`.
- Ноль - код завершения процесса, а не доказательство работоспособности сайта.

После `start` и повторных проверок:

- HTTP 200 и `Welcome to nginx!` восстановились.
- `ps` показывает `Up`; ID остался `0e4c538a2d84`.
- Изменилось время работы, но не сам экземпляр контейнера.

Почему нужен `start`, а не `run`

`start` запускает существующий остановленный объект. `run` создаёт новый; занятое имя приведёт к конфликту. У каждого контейнера уникальный ID, назначенный Docker, и уникальное имя в пределах Engine.

При `stop` сохраняются конфигурация и записываемый слой. При удалении контейнера этот слой удаляется; образ остаётся отдельным объектом. Хранение важных данных через `volumes` - следующая отдельная тема, не выполненная часть этой лабы.

Защита: что получилось

Первый вопрос про три сущности пришлось уточнить: образ не равен контейнеру. После объяснения я верно указал, что два контейнера с разными ID можно останавливать независимо. Второй ответ точный: 8133 - Ubuntu, 80 - контейнер, `curl` обращается к `127.0.0.1:8133`.

Состояние на завершение

Я согласился сохранить контейнер остановленным для следующей практики. Команда финального `stop` предложена, но подтверждения её выполнения после последнего `start` нет. Последний показанный статус - `Up`. В реферате не утверждается, что контейнер сейчас остановлен или удалён.

Вывод

Запуск образа - начало, а не приёмка. Нужна согласованность состояния контейнера, публикации, ответа клиента и журнала. Остановка и старт одного ID демонстрируют сохранение объекта, а не создание нового.

04 / Команды по классам

Практическая шпаргалка Docker

Доступ к Engine / ПРОСМОТР

```
sudo docker version
```

Смотри Client и Server. Только Client плюс ошибка - не доказательство, что Engine отсутствует. В нашей практике доступ обеспечен через sudo.

Инвентаризация / ПРОСМОТР

```
sudo docker ps
sudo docker ps -a
```

Без -a видны работающие контейнеры; -a добавляет остановленные. Поля: ID, IMAGE, STATUS, PORTS, NAMES. В Session 13 сравнивали один ID до и после stop/start.

Создание / ИЗМЕНЕНИЕ

```
sudo docker run -d --name <имя> \
  -p <IP_хоста>:<порт_хоста>:<порт_контейнера> <образ:тег>
```

-d: фон; --name: имя; -p: публикация. Формула не содержит буквальных угловых скобок в рабочей команде. Может скачать образ и создать сетевые настройки. Фактический пример - на странице 1.

Логи / ПРОСМОТР и HTTP / ПРОБА

```
sudo docker logs --tail 20 s18-web
curl -i --max-time 3 http://127.0.0.1:8133/
```

--tail 20 ограничивает последние строки; ищи свой GET и 200, а не только стартовые сообщения. curl -i показывает заголовки, --max-time ограничивает время. Запрос может создать запись в access log.

Жизненный цикл / ИЗМЕНЕНИЕ

```
sudo docker stop s18-web
sudo docker start s18-web
```

stop останавливает процесс, start запускает сохранённый контейнер. После каждой команды проверяй ps и исходный HTTP-запрос; не считай напечатанное имя полноценной приёмкой.

Очистка / УДАЛЕНИЕ / только обсуждалось

```
sudo docker rm s18-web
```

Для согласованного удаления остановленного контейнера, не выполнять сейчас. Удаляется объект и его записываемый слой, но не образ. В этой практике удаление не подтверждено. rgube я не использовал.

05 / Повторение и границы навыка

Для блокнота

- Образ - шаблон; контейнер - созданный экземпляр; процесс может быть остановлен.
- `run` создаёт новый объект. `start` запускает старый. `stop` не удаляет.
- ID позволяет проверить, что экземпляр сохранился.
- `HOST_PORT` и `CONTAINER_PORT` - разные стороны публикации.
- `Up` не заменяет HTTP-код, содержимое страницы и журнал запроса.
- Конкретный тег полезен для версии; `digest` точнее фиксирует содержимое.
- Ошибка `socket permissions` не равна «Docker не установлен».
- После учебной проверки не оставляю ненужные процессы без решения об их судьбе.

Самопроверка без терминала

1. Где хранится работающий процесс: в образе или в созданном контейнере?
2. Что останется после `stop`? Что будет потеряно при `rm` без внешнего хранения?
3. Почему повторный `run` с тем же именем не заменяет `start`?
4. Какие адрес и порт использует `curl` в нашем SSH-сеансе?
5. Почему такой же `localhost URL` на Mac не обязан работать?
6. Как докажешь, что ответ пришёл от нужного контейнера?
7. Почему доступ к группе `docker` нельзя выдавать как ограниченные права на один сервис?

Следующая практика

Разграничение пользователей, групп и `sudo`. План: отдельные роли с положительными и отрицательными проверками доступа. На дату занятия это был план. Обновление 01.10: я создал `dev/view/ops` и `lab`, проверил `dev:lab 2750; sudoers ops` и отдельные SSH-входы ещё не подтверждены. Не обещаем изоляцию контейнерных полномочий одной `docker`-группой.

Источники и доказательства

Основное evidence - мои скриншоты от 30 сентября: `version`, пустой `ps`, `run/pull`, `HTTP/ps`, `logs`, `stop/Exited`, повторный `HTTP/ps`. Это учебная работа с пояснениями команд, не независимый production-кейс.

- Docker run: <https://docs.docker.com/reference/cli/docker/container/run/>
- Docker start: <https://docs.docker.com/reference/cli/docker/container/start/>
- Публикация портов: <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- Доступ к daemon: <https://docs.docker.com/engine/install/linux-postinstall/>

В теории упоминались `inspect`, `port`, `image ls` и `pull` как отдельная команда. На показанных шагах они отдельно не выполнялись: не включаю их в свою подтверждённую практику. Образ скачан автоматически через `run`.

Автоматического `check-s13` в этой практике нет: приёмка по наблюдениям и защите.